

Doc. No.: WG21/P0943R1

Date: 2018-03-30

Reply-to: Hans-J. Boehm

Email: hboehm@google.com

Authors: Hans-J. Boehm

Audience: LEWG

P0943R1: Support C atomics in C++

As we previously suggested in [P0063R0](#), it would be nice to have a real interoperability story for C atomics and C++ atomics. In the interest of time, and since it appeared to be less essential and more controversial than the rest of that proposal, this suggestion was not pursued further in later revisions of that paper. Nonetheless, it remains a gaping hole in the interoperability story between C and C++. The solution proposed here is based on one that has been used by Android for several years.

The Problem

It is desirable to make C language header files directly usable by C++ code. It is increasingly common for such header files to include declarations that rely on atomics.

For example, a header may wish to declare a function that provides saturating ("clamped") addition on atomic integers. This can easily be implemented in the common subset of C and C++, except that we have no convenient and portable way to include the definition of names such as `atomic_int`, that are defined in `<atomic>` in C++ and in `<stdatomic.h>` in C.

The best we can currently do in portable code is to use an explicit preprocessor test followed by conditional inclusion of either `<atomic>` or `<stdatomic.h>`. In the former case, we then have to add using declarations to inject the required types and functions into the global namespace. Thus we end up with something like

```
#ifdef __cplusplus
    #include <atomic>
    using std::atomic_int;
    using std::memory_order;
    using std::memory_order_acquire;
    ...
#else /* not __cplusplus */
    #include <stdatomic.h>
#endif /* __cplusplus */
...
int saturated_fetch_add(atomic_int *loc, int value, memory_order order);
```

This approach relies on the currently implicit assumption or wish that the representation of C and C++ atomics are compatible.

Although certainly possible, this is very clumsy compared to the level of interoperability we normally provide; for other language features we commonly provide a C header file that can be included from C++, and provides the necessary declarations. Here we propose to do the same for atomics.

The story here is somewhat confused by the fact that most of the other `.h` compatibility headers currently only appear in the deprecated features section under D.5 C standard library headers [depr.c.headers]. A recent paper, [P0619](#) proposes to undeprecate them. We strongly agree with this paper that the current deprecation of these headers does not reflect reality. Many of them are widely used and heavily relied upon. This proposal assumes that at least some of those headers will be preserved.

History

C and C++ atomics were originally designed together. The original design ensured that the non-generic pieces of the C++ atomics library were usable as a C library. This interoperability story became less clear as a result of several later decisions:

- WG14 adopted, in addition to the non-generic parts of the C++ library, a more generic facility, relying on the `_Atomic` type qualifier, and the `_Atomic(T)` type specifier. This happened late enough in the C++11 standardization cycle that there wasn't much of an opportunity for C++ adjustments.
- C++ has never defined what it means to include `stdatomic.h` in a C++ program.

Shortly before finalizing C++11, we briefly discussed accommodating `_Atomic` roughly as described here.

Proposal

We propose to add a header `stdatomic.h` to the C++ standard. This would mirror the identically named C header, and provide comparable functionality. However, instead of defining it in terms of the C header, we propose that it have the following effects:

1. Include the `<atomic>` header.
2. Define a macro `_Atomic(T)` as `std::atomic<T>`.
3. Promote all `atomic_...` and `memory_order...` identifiers introduced by the `<atomic>` header into the global name space.

Although this provides functionality very similar to the C header, it is unlikely that it will be exactly the same. As usual, it is the responsibility of the author of a C and C++ shared header to ensure that the code is correct in both languages.

This proposal makes no effort to support the C `_Atomic` type qualifier, as opposed to the type *specifier* (which is spelled with parentheses). This is intentional; accommodating the qualifier would be a major language change, particularly since it is the only type qualifier that can affect alignment.

We believe this is the minimum required to conveniently use atomics in shared headers.

Implementation compatibility

This proposal reflects current practice in Android since the Android Lollipop release in 2014. (See <https://android.googlesource.com/platform/bionic/+lollipop-release/libc/include/stdatomic.h>. That uses a single include file shared between C and C++, which may not be the optimal implementation strategy.)

In an ideal world, this would be easy to support in various compiler environments. The main requirement is that C and C++ atomics should be implemented identically and use the same ABI conventions. There is no fundamental reason to do anything differently. Implementing ordering constraints in incompatible ways is already greatly undesirable, in that mixed language programs with `memory_order` annotations would no longer be sequentially consistent. Using incompatible locking schemes for large atomics would generally double the amount of space required for the lock table, and complicate ABI conventions, with no benefit.

Our preliminary experiments with gcc-4.9 found no differences on x86-64, though it did find some seemingly gratuitous alignment differences on ARMv7. Unfortunately later information from Jonathan Wakely and David Goldblatt points out that gcc in fact relies on two separate mechanisms for determining atomics alignment in C and C++, and these differ in some corner cases, since alignment decisions are made by the compiler in one case and the library the other. Although this is nontrivial to change, and requires C++ ABI changes in those corner cases, those changes appear desirable, even independent of this proposal. Some applications already assume compatible C++ and C ABIs for atomics, and those will currently break. And the current approach appears to preclude a consistent ABI between e.g. gcc and clang. There appears to be general agreement that this should be fixed in any case. More discussion can be found at https://gcc.gnu.org/bugzilla/show_bug.cgi?id=71660.

Thus the implementation effort for meaningful conformance to this proposal will vary between adding a simple header file on one hand, and appreciable changes to the existing atomics alignment logic in either C or C++ on the other. The implementation effort for minimal conformance consists of only the header file, since we can't normatively say anything about C and C++ consistency.

Should there be a `<cstdatomic>`?

We do not believe there is a strong need to introduce a `<cstdatomic>` that only introduces names into the `std::` namespace. The justification for `<stdatomic.h>` is entirely to support headers shared between C and C++, not to import a C facility into C++. A shared header will not be able to take advantage of names introduced into the `std::` namespace.

There is an argument that `<cstdatomic>` should be provided anyway for uniformity. It's not clear that this outweighs the cost of introducing a header that we expect to get essentially no use. There did not appear to be sentiment in SG1 for adding `<cstdatomic>`.

Wording

Placement and details of the wording are heavily dependent on the outcome of the P0619 discussion. We expect something along the following lines:

The header `<stdatomic.h>` behaves as if it consisted of:

`#include <atomic>`

```
#define Atomic(T) std::atomic<T>
```

```
using std::memory_order;  
using std::memory_order_relaxed;  
using std::memory_order_consume;  
using std::memory_order_acquire;  
using std::memory_order_release;  
using std::memory_order_acq_rel;  
using std::memory_order_seq_cst;
```

```
using std::atomic_flag;
```

```
using std::atomic_bool;  
using std::atomic_char;  
using std::atomic_schar;  
using std::atomic_uchar;  
using std::atomic_short;  
using std::atomic_ushort;  
using std::atomic_int;  
using std::atomic_uint;  
using std::atomic_long;  
using std::atomic_ulong;  
using std::atomic_llong;  
using std::atomic_ullong;  
using std::atomic_char16_t;  
using std::atomic_char32_t;  
using std::atomic_wchar_t;  
using std::atomic_int8_t;  
using std::atomic_uint8_t;  
using std::atomic_int16_t;  
using std::atomic_uint16_t;  
using std::atomic_int32_t;  
using std::atomic_uint32_t;  
using std::atomic_int64_t;  
using std::atomic_uint64_t;  
using std::atomic_int_least8_t;  
using std::atomic_uint_least8_t;  
using std::atomic_int_least16_t;  
using std::atomic_uint_least16_t;  
using std::atomic_int_least32_t;  
using std::atomic_uint_least32_t;  
using std::atomic_int_least64_t;  
using std::atomic_uint_least64_t;  
using std::atomic_int_fast8_t;  
using std::atomic_uint_fast8_t;
```

```
using std::atomic_int fast16 t;  
using std::atomic_uint fast16 t;  
using std::atomic_int fast32 t;  
using std::atomic_uint fast32 t;  
using std::atomic_int fast64 t;  
using std::atomic_uint fast64 t;  
using std::atomic_intptr t;  
using std::atomic_uintptr t;  
using std::atomic_size t;  
using std::atomic_ptrdiff t;  
using std::atomic_intmax t;  
using std::atomic_uintmax t;
```

The using declarations for intN t, uintN t, intptr t, and uintptr t are included only if the corresponding types are defined in the <atomic> header.

[Note: Implementations should ensure that C and C++ representations of atomic objects are compatible, so that the same object can be safely accessed as both an Atomic(T) from C code and atomic<T> from C++ code. The representations should be the same, and the mechanisms used to ensure atomicity and memory ordering should be compatible. --end note]

The above is close to a verbatim reflection of the part of the Android <stdatomic.h> header that is used when compiling C++.

History

Changed in R1

Changes made to reflect SG1 discussion Jacksonville. SG1 voted 6/17/4/1/0 to advance to to LEWG, but suggested a few changes:

- Removed false claim in discussion that this is always easy to implement.
- Promote only types to global name space, and let ADL take care of the rest. Android promotes all of them, but this now seems suboptimal.
- Explicitly list all using declarations required for the namespace promotion.

Note that the SG1 discussion was slightly incorrect in that it overlooked the recent change of memory_order to an enum class. But in retrospect it's not at all clear that this affects our proposal. It simply adds one more easily avoidable incompatibility between C and C++ usages of the header.